Chapter 2 | Exercise solutions with Python

Recurrence and Encoder-Decoder Models

Exercise 1 follows a single number through three recurrent updates. Exercise 9 checks which source positions the encoder actually reads. Exercises 10 and 11 use one trained checkpoint to study the difference between scoring a reference and generating a sequence. The written solutions cover Exercises 2–8 and 12.

To rerun this notebook, open it inside a copy of `transformers-book` with a Python kernel that has the packages in `requirements-ch02.txt`. The model and its synthetic color sequences run on a CPU. The notebook includes the executed outputs if you only want to read the solutions.

The setup locates the Chapter 2 companion code from either the repository root or the notebook's folder. We use the same functions that the chapter examples use.

```
from pathlib import Path
import sys

import numpy as np
import torch

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    module = repo_root / "examples" / "ch02_recurrence" / "model.py"
    if module.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")

from examples.ch02_recurrence.data import PAD, collate
from examples.ch02_recurrence.model import RecurrentSeq2Seq, rnn_trace
```

1. Follow a state by hand

Read the inputs `0.2`, `0.4`, and `-0.1` in that order. At each position the network receives the new number and the state carried from the previous position. The state is one number in this example. It starts at zero.

For input x_i, first form the weighted sum $a_i = 0.5h_{i-1} + x_i$. Then apply the hyperbolic tangent, written $\tanh$, to get the next state $h_i = \tanh(a_i)$. The function keeps the state between -1 and 1. Use the calculator's full stored result as h_{i-1} in the next row. The values in the right column are the six-decimal answers to write down.

Position	Current input x_i	Weighted sum before $\tanh$	State to six decimals
1	0.2	$0.5(0) + 0.2 = 0.2$	0.197375
2	0.4	$0.5(0.197375320224904\ldots) + 0.4 \approx 0.498687660112452$	0.461084
3	−0.1	$0.5(0.461084444830037\ldots) - 0.1 \approx 0.130542222415019$	0.129806

At position 1 the old state contributes zero. At position 2 it contributes about 0.098688 on top of the new input. At position 3 the new input is negative, but half the previous state is about 0.230542. The weighted sum and the new state therefore stay positive. This is how the earlier inputs can affect the answer at a later position.

```
inputs = [0.2, 0.4, -0.1]
states = rnn_trace(inputs, recurrent_weight=0.5, initial_state=0.0)
expected = [0.197375320224904, 0.461084444830037, 0.129805707303656]

np.testing.assert_allclose(states, expected, rtol=0, atol=1e-14)
for position, state in enumerate(states, start=1):
    print(f"h_{position} = {state:.6f}")
```

```
OUTPUT
h_1 = 0.197375
h_2 = 0.461084
h_3 = 0.129806
```

`rnn_trace` returns the state after each input, so its first element is h_1. Formatting a number with `:.6f` changes its printed representation. The model keeps the full computed value for the next update, and the assertion compares the values before formatting.

The input x_i and the incoming state h_{i-1} can change at each position. The weighted sum and outgoing state change with them. The input weight 1, recurrent weight 0.5, and bias 0 are shared across positions. The initial state $h_0 = 0$ is the starting condition. After the first position, the next update uses the state produced by the previous one.

9. Test padding

Consider the source *red green blue*. Its three real tokens can be stored in a row of width 3 or in a row of width 6 with three extra `PAD` entries on the right. In both cases the encoder should stop after *blue*. The separate length array says where that happens.

Packing makes the recurrent encoder run only over the valid prefix of each row. We therefore expect the same final hidden state and cell state for both inputs. We also expect the same decoder logits when we use those states to start the decoder and give it the same shifted reference tokens. We fix the model weights for both calls. Untrained weights are enough to check this property of the computation.

```
padding_batch = collate([(3, 4, 5)])  # red, green, blue
source = padding_batch.source
true_lengths = padding_batch.lengths
decoder_input = padding_batch.decoder_input
extra_padding = torch.full((1, 3), PAD, dtype=source.dtype)
padded_source = torch.cat((source, extra_padding), dim=1)

with torch.random.fork_rng(devices=[]):
    torch.manual_seed(2026)
    padding_model = RecurrentSeq2Seq().double().eval()

print("Original source:", source.tolist())
print("Extended source:", padded_source.tolist())
print("True length for both:", true_lengths.tolist())
print("Decoder inputs for both:", decoder_input.tolist())
```

OUTPUT

```
Original source: [[3, 4, 5]]
Extended source: [[3, 4, 5, 0, 0, 0]]
True length for both: [3]
Decoder inputs for both: [[1, 5, 4, 3]]
```

Compare whole tensors. The hidden and cell states each have shape `(1, 1, 64)` for one layer and one example. The decoder reads `BOS blue green red` and scores nine vocabulary entries at each of four positions, giving logits of shape `(1, 4, 9)`. Use evaluation mode and keep gradient recording off. In the comparison below, `assert_close` checks all coordinates and the maximum absolute difference summarizes each comparison in one number.

```
with torch.inference_mode():
    hidden, cell = padding_model.encode(source, true_lengths)
    padded_hidden, padded_cell = padding_model.encode(
        padded_source, true_lengths
    )
    logits = padding_model(source, true_lengths, decoder_input)
    padded_logits = padding_model(
        padded_source, true_lengths, decoder_input
    )

comparisons = [
    ("Hidden state", hidden, padded_hidden),
    ("Cell state", cell, padded_cell),
    ("Decoder logits", logits, padded_logits),
]
for name, original, extended in comparisons:
    torch.testing.assert_close(original, extended)
    max_difference = (original - extended).abs().max().item()
    print(f"{name:14} {tuple(original.shape)}  max Δ = {max_difference:.2e}")
```

OUTPUT

```
Hidden state   (1, 1, 64)  max Δ = 0.00e+00
Cell state     (1, 1, 64)  max Δ = 0.00e+00
Decoder logits (1, 4, 9)  max Δ = 0.00e+00
```

The three zero differences show that widening storage alone changes none of the values in this run. To see what packing prevents, call the same LSTM directly on the two dense embedding tensors. This comparison deliberately omits the true length. The second dense call now runs three extra recurrent updates, even though the embedding at each added position is a zero vector.

```
with torch.inference_mode():
    _, (dense_hidden, dense_cell) = padding_model.encoder(
        padding_model.source_embedding(source)
    )
    padded_embeddings = padding_model.source_embedding(padded_source)
    _, (dense_padded_hidden, dense_padded_cell) = (
        padding_model.encoder(padded_embeddings)
    )

assert torch.count_nonzero(padded_embeddings[:, 3:]) == 0
hidden_change = (dense_hidden - dense_padded_hidden).abs().max().item()
cell_change = (dense_cell - dense_padded_cell).abs().max().item()
print(f"Without packing, max hidden change = {hidden_change:.6f}")
print(f"Without packing, max cell change   = {cell_change:.6f}")
```

```
OUTPUT
Without packing, max hidden change = 0.167324
Without packing, max cell change   = 0.325213
```

A zero embedding removes the contribution of that input vector. The previous hidden state and the biases can still change the gates. The cell can retain only a fraction of its old value or write a new contribution. Feeding zeros through the recurrence can therefore change both states, as this example shows. Packing skips those updates altogether.

The companion suite checks this rule in `test_packed_source_ignores_padding_and_restores_batch_order` in `tests/test_recurrence.py`. That test replaces the stored entries past each true length with a color ID. It also compares each batched state to the state from encoding its trimmed sequence on its own. These checks show that the encoder uses the true lengths and returns the states in the original batch order. Here we additionally varied the stored width and checked the downstream logits.

10. Separate scoring from generation

Take the source *red green blue*. Its reference target is *blue green red* followed by `<eos>`. Imagine that the decoder picks *yellow* at its first step. On the second step, **teacher forcing** feeds the reference token *blue*. **Greedy generation** feeds *yellow*, the token the model actually chose. The decoder now gets different input even though both runs began with the same source and weights.

We will look for that difference in a real held-out case. First, train the companion model for 750 updates on a CPU with seed 2026 and one thread. The program checks validation loss every 50 updates. Each check scores the reference targets with their correct prefixes. The checkpoint with the smallest validation token loss supplies the weights for both exercises. The test sequences play no part in adjusting the weights or selecting the checkpoint.

The training function writes its selected weights to `best.pt`. We reload that file and retain the model and metrics in memory. The temporary directory keeps this run separate from any existing book experiment. The following cell captures the progress log and prints the environment and the selected update.

```
from contextlib import redirect_stdout
from io import StringIO
from itertools import zip_longest
from tempfile import TemporaryDirectory

from examples.ch02_recurrence.data import BOS, EOS, VOCAB
from examples.ch02_recurrence.data import make_splits, words
from examples.ch02_recurrence.model import greedy_decode
from examples.ch02_recurrence.run_examples import run_training

training_seed = 2026
with TemporaryDirectory(prefix="ch02-solutions-") as directory:
    with redirect_stdout(StringIO()):
        training_run = run_training(
            steps=750, seed=training_seed, output=Path(directory)
        )
    checkpoint = torch.load(
        Path(directory) / "best.pt",
        map_location="cpu",
        weights_only=True,
    )

trained_model = RecurrentSeq2Seq()
trained_model.load_state_dict(checkpoint["model"])
trained_model.eval()
assert checkpoint["selected_step"] == training_run["selected_step"]

environment = training_run["environment"]
print("Python:", environment["python"])
print("PyTorch:", environment["torch"])
print("NumPy:", environment["numpy"])
print("CPU threads:", environment["threads"])
print("Selected update:", training_run["selected_step"])
```

```
OUTPUT
Python: 3.11.14
PyTorch: 2.9.0
NumPy: 2.2.6
CPU threads: 1
Selected update: 600
```

Recreate the same three splits with the run's seed. To choose an example, scan the saved test predictions in split order and take the first that differs from the full reference. This rule can find an incorrect color, a premature ending, or a sequence that reaches the generation cap without ending. If the selected checkpoint happens to solve every test case on a rerun, inspect the first test source with a seeded, untrained model. The held-out scores in Exercise 11 still come from the trained checkpoint.

```
train_set, validation_set, test_set = make_splits(seed=training_seed)
test_metrics = training_run["test"]
scored_cases = zip(test_set, test_metrics["predictions"])
case_index = next(
    (j for j, (source, prediction) in enumerate(scored_cases)
     if prediction != list(reversed(source)) + [EOS]),
    None,
)

case_model = trained_model
if case_index is None:
    case_index = 0
    torch.manual_seed(training_seed)
    case_model = RecurrentSeq2Seq().eval()
    print("All trained cases matched. Inspecting untrained weights.")

source = test_set[case_index]
case = collate([source])
print(f"Held-out item {case_index + 1} of {len(test_set)}")
print("Source:", words(source))
```

OUTPUT

```
Held-out item 14 of 48
Source: green blue black red red red
```

The batch contains one source, so its target row has no storage padding. Call the model with `case.decoder_input` to compute teacher-forced logits. The final axis holds one score per vocabulary entry. `argmax(dim=-1)` gives the ID of the highest-scoring token at each reference position. Softmax would preserve the order of the scores, so the largest logit also has the largest probability.

Next, call `greedy_decode` with the source, its length, and the cap of seven new tokens. It starts with `<bos>` and feeds back its own choices. It never receives `case.decoder_input` or `case.target`. Keep an emitted `<pad>` or `<bos>` if it appears before the first `<eos>`. Those are actual errors. This is a one-row batch, so when it produces `<eos>` the call stops immediately. For the teacher-forced choices, print the argmax at **every** reference position, even if it picks `<eos>` early. That pass advances using the reference inputs.

```
generation_cap = training_run["configuration"]["generation_cap"]
with torch.inference_mode():
    scores = case_model(case.source, case.lengths, case.decoder_input)
    greedy = greedy_decode(
        case_model, case.source, case.lengths, generation_cap
    )

reference_ids = case.target[0].tolist()
forced_choices = scores.argmax(dim=-1)[0].tolist()
greedy_choices = greedy[0].tolist()
assert reference_ids == list(reversed(source)) + [EOS]
if case_model is trained_model:
    assert greedy_choices == test_metrics["predictions"][case_index]

print("Reference:     ", words(reference_ids))
print("Teacher forced:", " ".join(VOCAB[i] for i in forced_choices))
print("Greedy:        ", words(greedy_choices))
```

OUTPUT

```
Reference:      red red red black blue green <eos>
Teacher forced: red red red black green green <eos>
Greedy:         red red red black green blue <eos>
```

To understand the first error, line up the two sets of decoder inputs with their outputs. The teacher-forced inputs are already in the batch. The greedy inputs are `<bos>` followed by all generated tokens except the last one. Each input after the first step was the previous step's greedy choice. If greedy decoding stops early, there is no greedy input for the remaining reference positions. A `--` marks such an absent entry.

```
forced_inputs = case.decoder_input[0].tolist()
greedy_inputs = [BOS] + greedy_choices[:-1]

def label(token_id):
    return "--" if token_id is None else VOCAB[token_id]

heading = ("Step  Reference  Forced in  Forced out  "
           "Greedy in  Greedy out")
print(heading)
timeline = zip_longest(
    reference_ids, forced_inputs, forced_choices,
    greedy_inputs, greedy_choices,
)
first_error = None
for step, tokens in enumerate(timeline, start=1):
    ref, forced_in, forced_out, greedy_in, greedy_out = map(label, tokens)
    print(f"{step:>4}  {ref:<9}  {forced_in:<9}  {forced_out:<10}  "
          f"{greedy_in:<9}  {greedy_out}")
    if first_error is None and ref != greedy_out:
        first_error = step

if first_error is not None:
    print(f"\nFirst greedy mismatch: step {first_error}")
    if first_error < min(len(forced_inputs), len(greedy_inputs)):
        forced_next = label(forced_inputs[first_error])
        greedy_next = label(greedy_inputs[first_error])
        print(f"Next inputs: forced={forced_next}, "
              f"greedy={greedy_next}")
    else:
        print("There is no next step active in both runs.")
```

OUTPUT

```
Step  Reference  Forced in  Forced out  Greedy in  Greedy out
   1  red        <bos>      red         <bos>      red
   2  red        red        red         red        red
   3  red        red        red         red        red
   4  black      red        black       red        black
   5  blue       black      green       black      green
   6  green      blue       green       green      blue
   7  <eos>      green      <eos>       blue       <eos>

First greedy mismatch: step 5
Next inputs: forced=blue, greedy=green
```

In the saved run, both passes chose the correct colors through step 4. At step 5 both had received *black* and both chose *green*, though the reference was *blue*. For step 6, teacher forcing supplied *blue*, and the decoder chose the reference token *green*. Greedy decoding supplied its own step-5 choice *green*, and the decoder chose *blue*. The two LSTM computations had shared a history through step 5. At step 6 they updated the state with different token embeddings.

Both passes selected <eos> at step 7 in this run. Stopping at the right time did not repair the swapped colors. A teacher-forced argmax sequence combines choices made with reference prefixes, including after errors. Greedy decoding instead commits to its own prefix. This is why a count of individually correct teacher-forced predictions answers a different question from the count of fully correct generations. The output table identifies the actual divergence on a rerun.

11. Read a length experiment carefully

A six-color source needs seven predictions, one per color in reverse order and one for <eos>. A generation is an **exact match** only when it equals that whole reference. An extra token, a color in the wrong position, or a missing ending makes the sequence incorrect. The denominator for a length group is the number of held-out source sequences of that length.

The same 750-update run supplies the scores below. The evaluator generated from each test source and compared the resulting token sequence, through the first ending if present, with the complete reference. It kept any incorrect <pad> or <bos> before the ending. We extract the counts by source length and divide matches by examples for each group. We also show the training and validation group sizes to identify which lengths the model saw before testing.

```
from collections import Counter

training_lengths = Counter(map(len, train_set))
validation_lengths = Counter(map(len, validation_set))
test_lengths = Counter(map(len, test_set))

print("Length  Train  Validation  Test  Exact matches   Rate")
for length in sorted(test_lengths):
    group = test_metrics["by_length"][str(length)]
    count = group["sequences"]
    exact = group["exact_matches"]
    assert count == test_lengths[length]
    rate = exact / count
    print(f"{length:>6}  {training_lengths[length]:>5}  "
          f"{validation_lengths[length]:>10}  {count:>4}  "
          f"{exact:>5}/{count:<7}  {rate:>6.1%}")

assert sum(test_lengths.values()) == test_metrics["sequences"]
print("Overall exact:", test_metrics["exact_matches"],
      "of", test_metrics["sequences"])
print("Generations ending in EOS:", test_metrics["ended_in_eos"])
print("Teacher-forced correct positions:",
      test_metrics["teacher_forced_correct_tokens"],
      "of", test_metrics["nonpadding_tokens"])
```

```
OUTPUT
Length  Train  Validation  Test  Exact matches   Rate
     3     48          12    12     12/12       100.0%
     4     48          12    12     12/12       100.0%
     5     48          12    12     12/12       100.0%
     6     48          12    12      7/12        58.3%
Overall exact: 43 of 48
Generations ending in EOS: 48
Teacher-forced correct positions: 258 of 264
```

In the saved run, the model matched 12 of 12 test examples at each of lengths 3, 4, and 5. It matched 7 of 12 at length 6, or about 58.3%. That gives 43 of 48 complete matches overall. All 48 generated sequences ended, so the five failures were still wrong even though each produced <eos>. The 258 correct teacher-forced decisions out of 264 are counted over token positions, including the endings. The 43 exact greedy matches out of 48 are counted over entire sequences.

Each complete test source is held out. For example, a particular six-color ordering can be new even if all its colors and its length were present in training. The model trained on 48 distinct sequences **at every test length**, including length 6. Thus the table measures generalization to new color arrangements at familiar lengths. The drop in this length-six group is a result for 12 examples with one seed and checkpoint. It does not establish a universal length limit or identify whether the errors came from state width, optimization, or decoding.

A separate length-extrapolation split. Decide the length sets and sample counts before training. Keep the same six colors, allow repeated colors within a source, and sample distinct complete source tuples with a fixed seed.

Split	Source lengths	Sequences per length	Total
Training	3, 4, 5	64	192
Validation	3, 4, 5	16	48
Test	6, 7, 8	16	48

At each of lengths 3 through 5, draw 80 different tuples and assign 64 to training and the other 16 to validation. For lengths 6 through 8, draw only test tuples. This keeps the held-out lengths out of both fitting and checkpoint selection. The validation score still selects the checkpoint. Evaluate the selected model once on the three longer groups and report matches and totals separately for each length.

Use a common generation cap of **9 new tokens** in validation and testing. The longest source has eight colors, so its correct target needs eight colors plus <eos>. A cap of seven would prevent a perfect reversal of that source from ending correctly. With a cap of nine, require the same full reference and ending for each length. A model that uses the whole allowance without emitting an ending still fails that example. This split is a proposed design. The measured rates above come from the chapter's original split.